

Edge deployment: Plant health classification (MobileNet V3) on Raspberry Pi Zero 2 W

Yusuf Savaş

Advisor: Fikret Gürgen

June 5, 2026

TABLE OF CONTENTS

1. Introduction	2
2. Description	3
2.1. Hardware	3
2.2. Software	3
2.3. Use	3
2.4. Diagram	4
3. Requirements	6
3.1. Features	6
4. Designed System	7
4.1. Architectural Choices	7
4.2. Inference Pipeline	7
5. Implementation	9
5.1. Data Preparation	9
5.2. Training Pipeline	10
5.3. ONNX Export	11
5.4. On-device Runtime (C++)	12
5.5. Web Portal and Test Bench	13
6. Results	14
6.1. Reference Baseline (Dev PC)	14
6.2. On-Device Performance (Raspberry Pi Zero 2 W)	14
6.3. Final Classification Accuracy	14
6.4. Architectural Hardware Constraints	15
7. Conclusions and Future Work	16
7.1. Future work	16
REFERENCES	17

1. Introduction

Plant pests and diseases destroy up to **40%** of global crop production each year and cost the world economy about **US\$220 billion** annually [1]; major staple crops see **10–40%** yield losses to pathogens [2]. Catching disease early from a leaf image enables timely treatment, less blanket pesticide use, and better yield protection, especially for smallholders who rarely have fast access to a pathologist.

We build an **edge-deployed** classifier on a **Raspberry Pi Zero 2 W** [3]: a live camera frame is strictly labeled **healthy**, **diseased**, or **background** (non-leaf) using **MobileNet V3** [4]. The model is exported to **ONNX** [5] and executed via **ONNX Runtime** [6] in a highly memory-efficient **C++** pipeline designed for constrained hardware. The main focus points are:

- Train on public leaf and background data, then deploy the exact same preprocessing and graph natively on the Pi.
- Maximize throughput on resource-constrained hardware using a multi-threaded **FP32** pipeline.
- Expose real-time results through a **web portal** and validate with a **real camera**, a 3D-printed test bench, and field-style captures.

2. Description

2.1. Hardware

The **Raspberry Pi Zero 2 W** was chosen as a **low-cost, resource-constrained** edge node. Because it relies heavily on an older architecture without dedicated Neural Processing Units (NPUs) or modern integer math extensions, the pipeline is highly optimized for memory and standard floating-point operations.

Item	Detail
CPU	Quad-core ARM Cortex-A53 (~1 GHz, ARMv8.0-A)
RAM	512 MB
Inference	CPU only (no GPU or NPU)

2.2. Software

- **Model:** **MobileNet V3** [4] exported to **ONNX** [5] for deployment.
- **Runtime:** **ONNX Runtime** [6] executed via a native **C++** API to prevent memory bloat.
- **Pipeline:** **Camera capture** → **NEON-accelerated ImageNet normalization** → **FP32 ONNX Inference** → **Temporal smoothing** → **Output**.
- **Web UI:** A **web portal** on the local network displaying the **live preview**, **predictions**, and **FPS throughput**.

2.3. Use

In practice, the system continuously analyzes frames from the Pi Camera. For a **demo**, an operator opens the **web page** on a smartphone or laptop sharing the same Wi-Fi. By presenting **healthy** and **diseased** plants to the camera, the operator can view real-time classifications, model confidence, and system health metrics (FPS, CPU temperature). The background class enables the system to correctly reject non-leaf

objects automatically.

2.4. Diagram

Figure 2.1 shows the logical data flow.

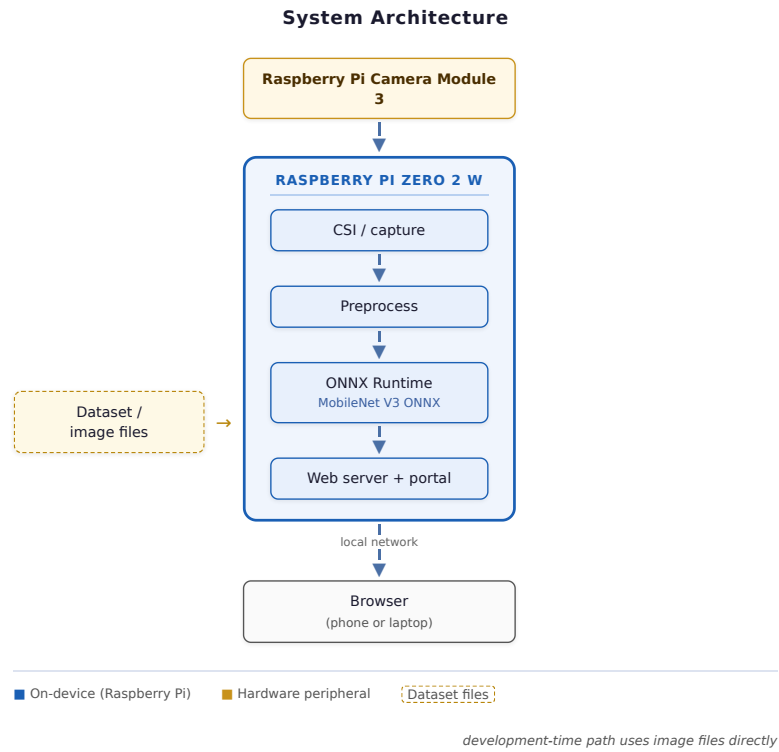


Figure 2.1. System architecture: camera and files into preprocess, ONNX Runtime, web portal, and browser on the LAN.

3. Requirements

3.1. Features

- Provide a lightweight **web portal** to visualize **inference results** and **per-class confidence**.
- Expose **performance metrics**: **FPS**, **latency**, and **thermal data** updated iteratively.
- Support both **live camera streams** and **batch evaluation** of static datasets using the exact same C++ memory stack.

Layers

Layer	Role
Presentation	Web UI serving the MJPEG live stream, prediction status, and session summary.
Application / API	Serves the UI and pushes inference metrics (Web-Socket/HTTP).
Inference	Evaluates the MobileNet V3 graph using ONNX Runtime directly on the CPU.
Acquisition	Captures frames via libcamera or loads testing batches, feeding them to the preprocessor.

4. Designed System

4.1. Architectural Choices

- **MobileNet V3** [4] was selected because its depthwise separable convolutions offer a strong balance of accuracy and parameter efficiency (**1,519,906** parameters) for edge devices.
- **3-Class Topology**: Training strictly on a binary "healthy vs. diseased" split forces the model to produce false positives when viewing empty backgrounds. Expanding the architecture to include a generalized **background** class resolves this cleanly without requiring secondary OpenCV gating loops.
- **FP32 Over INT8**: While integer quantization (INT8) is generally preferred on the edge, the Pi Zero 2 W's Cortex-A53 lacks modern SDOT and UDOT hardware instructions. Consequently, INT8 operations fall back to slower software routines, incurring significant Quantize/Dequantize (QDQ) overhead. Therefore, a multi-threaded **FP32** pipeline utilizing mature NEON FMA instructions provides the highest throughput and preserves accuracy.
- **Native C++**: Avoiding Python in the deployment environment keeps the memory footprint well below the 512 MB ceiling, preventing Out-Of-Memory (OOM) crashes.

4.2. Inference Pipeline

- (i) **Input**: Libcamera pulls a continuous RGB stream from the Camera Module 3.
- (ii) **Preprocess**: The C++ pipeline uses ARM NEON vector instructions to quickly resize the frame to 224×224 and normalize it against standard ImageNet statistics, formatting it strictly into an NCHW tensor buffer.
- (iii) **Inference**: ONNX Runtime executes the FP32 graph across 4 threads.
- (iv) **Post-Process**: Softmax and argmax isolate the top prediction. Exponential Moving Average (EMA) smoothing is applied across frames to eliminate momen-

tary visual flickering.

5. Implementation

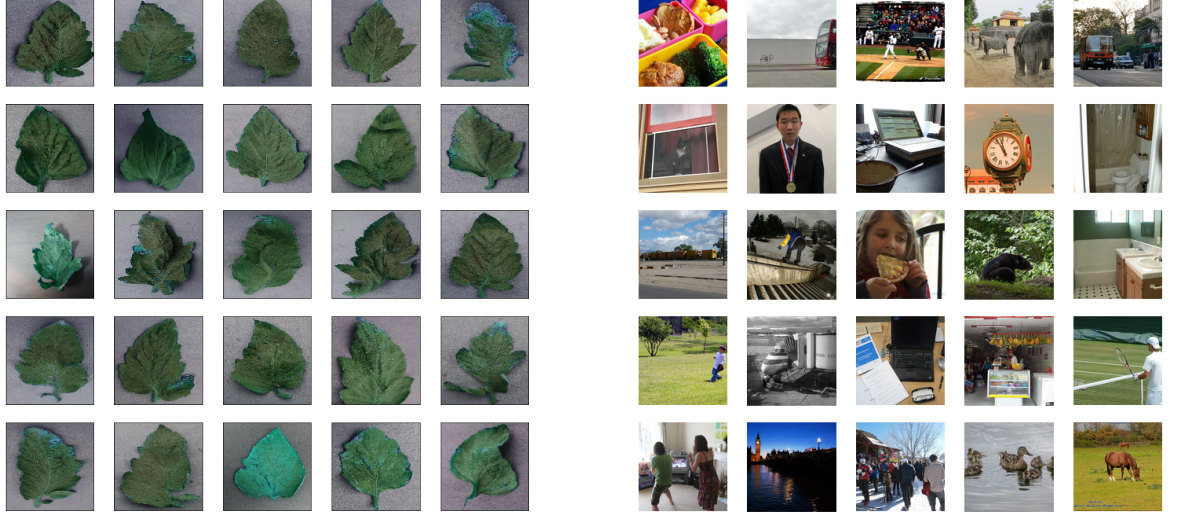
The system is developed in two discrete stages. The model is trained and exported on a CUDA-enabled development workstation. That frozen graph is then deployed to the native C++ runtime on the Raspberry Pi.

5.1. Data Preparation

The classifier operates on a three-class dataset: **healthy**, **diseased**, and **background**.

The leaf imagery relies on **PlantVillage** [7]. The original 38 fine-grained crop/disease labels are collapsed into a robust binary health signal: healthy leaves become **healthy**, while affected leaves become **diseased**.

To prevent the model from confidently hallucinating diseases when presented with empty tables or hands, a **background** class was synthesized from **COCO 2017** [8] and field-capture negatives. The combined dataset is split **70 / 15 / 15**, cleanly separating training, validation, and testing logic.



(a) PlantVillage leaf samples (healthy / diseased)

(b) COCO 2017 background samples

Figure 5.1. Dataset sources. The non-leaf scenes provide the critical background class for real-world camera deployments.

5.2. Training Pipeline

The network leverages transfer learning, starting from a **MobileNet V3-small** [4] backbone pretrained on ImageNet. The original 1,000-class head is replaced by a custom 3-class linear head.

During the 15-epoch training loop, frames are augmented with horizontal flips, $\pm 15^\circ$ rotations, and color jitter. A weighted sampler ensures balanced batches across the three categories. Only the checkpoint with the lowest validation loss is preserved and evaluated against the held-out test split.

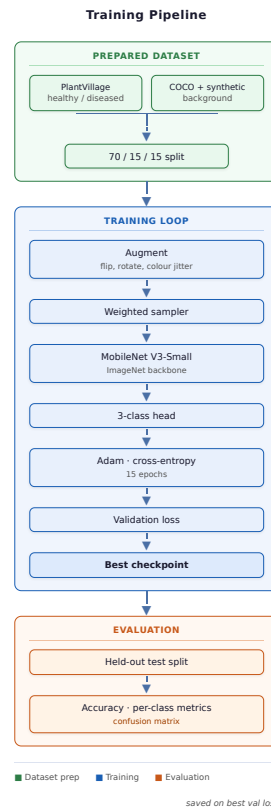


Figure 5.2. Training pipeline: from data augmentation through transfer learning into validation-based checkpointing.

5.3. ONNX Export

The optimal PyTorch model is converted to the **ONNX** [5] standard. This separation of concerns allows the Raspberry Pi to execute a frozen, mathematically identical graph without requiring heavy Python machine learning frameworks. The deployment graph expects a statically sized 224×224 input tensor and outputs three class logits.

5.4. On-device Runtime (C++)

The edge stack is programmed in **C++** using the **ONNX Runtime** [6] API. Memory operations are aggressively zero-allocation in the hot loop to maintain stability on 512 MB RAM.

Capture. The Pi Camera Module 3 feeds frames through the **libcamera** [9] stack.

Preprocessing. A fused routine resizes the image and normalizes it with ImageNet standard deviations. Using **NEON** vector instructions (`vfmaq_f32`), this routine prevents data-preparation overhead from bottlenecking the CPU.

Inference & Serving. ONNX Runtime manages the FP32 forward pass. An embedded HTTP server exposes the live UI, broadcasting an MJPEG preview and sending continuous Server-Sent Events (SSE) detailing predictions, smoothed probabilities, and hardware temperatures.

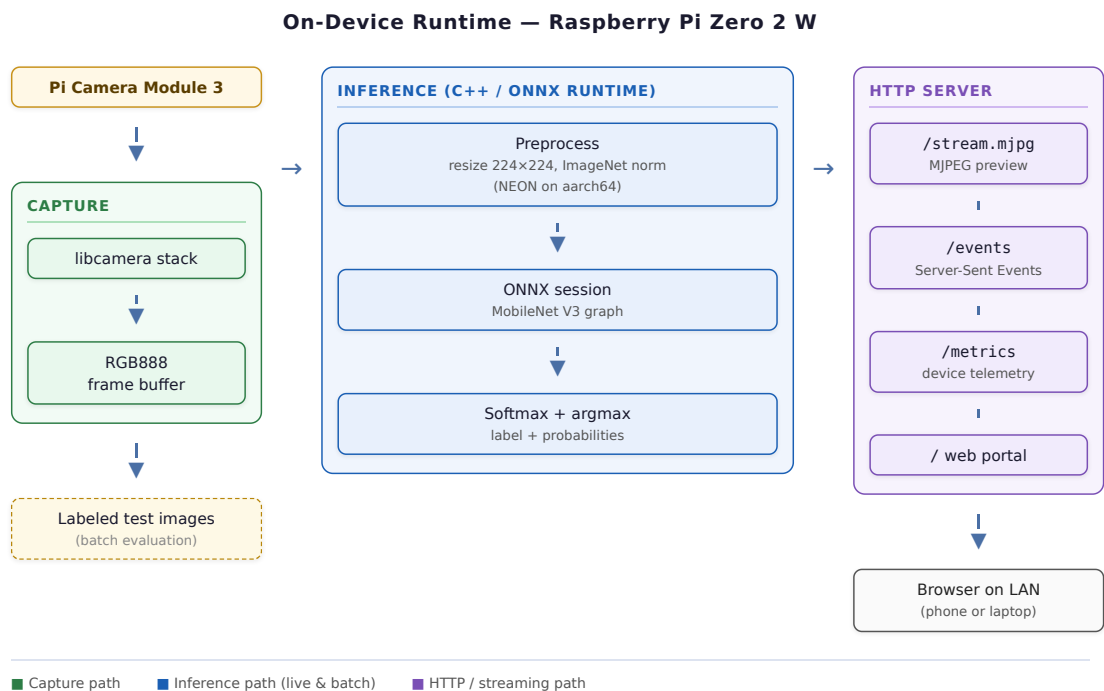
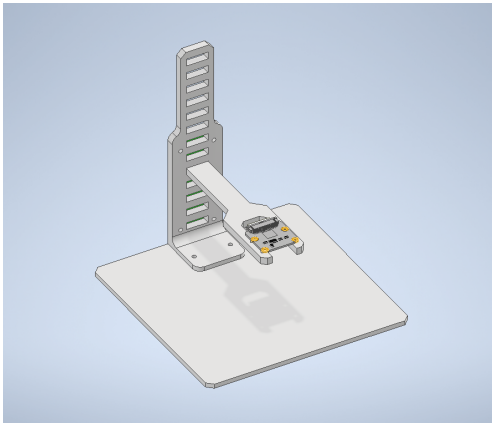


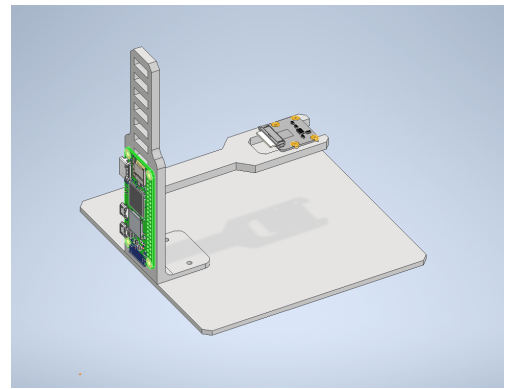
Figure 5.3. On-device runtime architecture executing on the Pi Zero 2 W.

5.5. Web Portal and Test Bench

The local web portal allows users to visualize real-time interactions with the model. To ensure repeatable, scientific validation outside of the dataset imagery, a 3D-printed PLA test bench was designed. It holds the camera at a strict height and angle, standardizing framing and lighting for physical leaf samples.



(a) CAD front



(b) CAD back

Figure 5.4. 3D-printed (PLA) test-bench camera stand, CAD model.

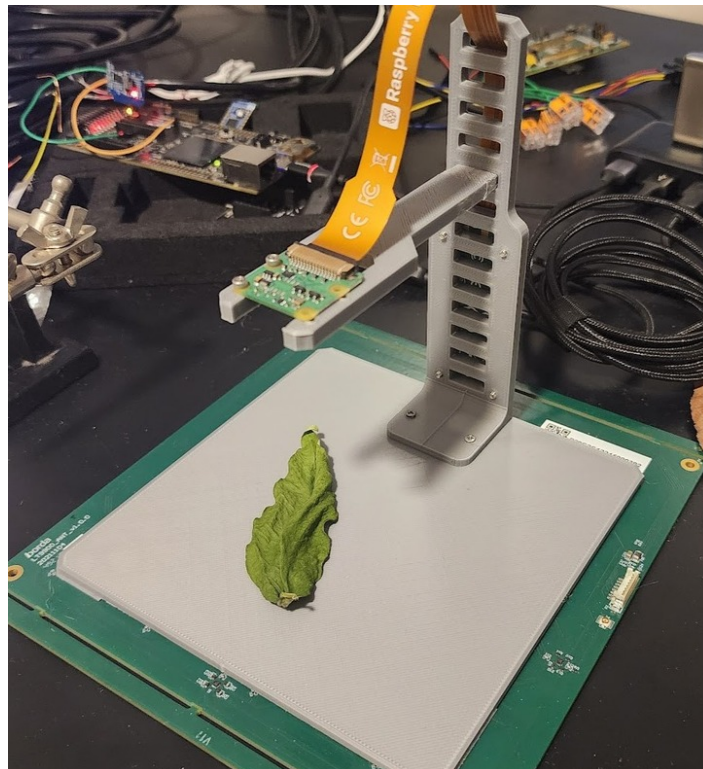


Figure 5.5. Printed test bench in use with the Pi camera and a leaf sample.

6. Results

This chapter reports inference benchmarks for the edge device (Raspberry Pi Zero 2 W running ONNX Runtime in native C++) compared against a development workstation reference (PyTorch on a Dev PC with an NVIDIA RTX 3050).

The edge deployments were tested using the complete 3-class dataset (12,606 images) to validate the final pipeline, alongside a legacy 2-class split (8,106 images) for baseline comparisons. Result only include the pipeline for processing the images, not encoding them for the web portal. So results are comparasion for the model and model implementation.

6.1. Reference Baseline (Dev PC)

Before moving to the edge, the model was evaluated on high-performance desktop hardware to establish a maximum theoretical baseline.

- **Accuracy:** 99.80%
- **Throughput:** 751.23 images/second

6.2. On-Device Performance (Raspberry Pi Zero 2 W)

The core objective was to balance accuracy and latency on the memory-constrained Raspberry Pi. We evaluated standard floating-point (FP32) execution against integer-quantized (INT8/QDQ) execution across different threading configurations.

6.3. Final Classification Accuracy

To illustrate the model’s reliability in its optimal edge state, the confusion matrix below details the exact predictions for the final **FP32 (3-class)** model. This model

Table 6.1. Edge Deployment Summary

Build Configuration	Split	Accuracy	Throughput
FP32 (1 thread)	2-class (8,106 images)	99.83%	8.03 img/s
FP32 (4 threads)	3-class (12,606 images)	99.81%	15.73 img/s
INT8 (1 thread)	3-class (12,606 images)	97.21%	6.04 img/s
INT8 (4 threads)	3-class (12,606 images)	97.05%	9.83 img/s

yielded highly balanced per-class F1 scores: Healthy (0.9948), Diseased (0.9984), and Background (0.9993).

Table 6.2. Confusion Matrix (FP32, 3-class)

Actual \ Predicted	Healthy	Diseased	Background
Healthy	2,215	5	2
Diseased	13	5,871	0
Background	3	1	4,496

6.4. Architectural Hardware Constraints

The benchmark results highlight a critical hardware reality regarding INT8 quantization on older architectures.

- **The FP32 Advantage:** The 4-thread FP32 configuration delivered the strongest performance, maintaining near-perfect accuracy (99.81%) while nearly doubling the throughput (15.73 images/second) compared to a single thread.
- **The INT8 Speed Penalty:** Despite being a quantization method designed for speed, INT8 tested roughly 33% slower than the FP32 baseline on a single thread. It also resulted in a 2.5–2.7% drop in accuracy.
- **Hardware Limitation:** The Pi Zero 2 W uses a Cortex-A53 CPU, which physically lacks the SDOT and UDOT hardware instructions required to accelerate integer dot-products. Without these instructions, INT8 is forced onto slower software fallback paths, while FP32 fully utilizes the processor’s mature NEON pipelines.

7. Conclusions and Future Work

We successfully developed and deployed a highly accurate 99.8% MobileNet V3 classification system natively on a Raspberry Pi Zero 2 W. By formatting the model into a 3-class architecture (Healthy, Diseased, Background), the pipeline robustly rejects non-leaf imagery without requiring secondary, memory-intensive detection models.

Using a native C++ runtime and carefully tuned multi-threading, end to end pipeline processes frames and serves results at nearly **8 images per second**. (Encoding latency is 31ms and inference latency is 68ms.) While INT8 quantization was aggressively tested during development, the specific Cortex-A53 processor of the Pi Zero 2 W lacks the hardware dot-product instructions (SDOT/UDOT) required to accelerate integer math. Consequently, the overhead of converting floats to integers inside the network makes INT8 slower than standard execution. Retaining the model in FP32 allowed the engine to leverage mature NEON FMA instructions, proving to be the optimal choice for this specific silicon.

Furthermore, when testing with real leaves in live environments, we observed that the camera’s field of view (FOV) and variations in background color could occasionally affect the boundary between healthy and diseased classifications. To mitigate this domain shift and achieve the most reliable edge performance, future iterations of the model should ideally be trained or fine-tuned on a dataset captured directly by the specific camera hardware used for deployment.

7.1. Future work

- **Field validations:** Expand testing beyond the 3D-printed bench into active greenhouses, exposing the model to natural, varied lighting over extended periods.
- **Calibration dynamics:** Implement confidence thresholds to further tune the boundary between diseased and healthy states based on seasonal plant variations.

REFERENCES

1. Food and Agriculture Organization of the United Nations (FAO), “Climate change fans spread of pests and threatens plants and crops, new FAO study”, <https://www.fao.org/newsroom/detail/Climate-change-fans-spread-of-pests-and-threatens-plants-and-crops-new-FAO-study>, 2021, accessed 2026.
2. Savary, S., L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts and A. Nelson, “The global burden of pathogens and pests on major food crops”, *Nature Ecology & Evolution*, Vol. 3, No. 3, pp. 430–439, 2019.
3. Raspberry Pi Ltd., “Raspberry Pi Zero 2 W”, <https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/>, 2021, accessed 2026.
4. Howard, A., M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le and H. Adam, “Searching for MobileNetV3”, *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019.
5. ONNX Community, “ONNX: Open Neural Network Exchange”, <https://onnx.ai>, 2019, accessed 2026.
6. ONNX Runtime developers, “ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator”, <https://onnxruntime.ai>, 2021, accessed 2026.
7. Hughes, D. P. and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics”, *arXiv preprint arXiv:1511.08060*, 2015.
8. Lin, T.-Y., M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár

and C. L. Zitnick, “Microsoft COCO: Common Objects in Context”, *European Conference on Computer Vision (ECCV)*, pp. 740–755, Springer, 2014.

9. libcamera developers, “libcamera: An open source camera stack and framework for Linux, Android, and ChromeOS”, <https://libcamera.org>, 2024, accessed 2026.